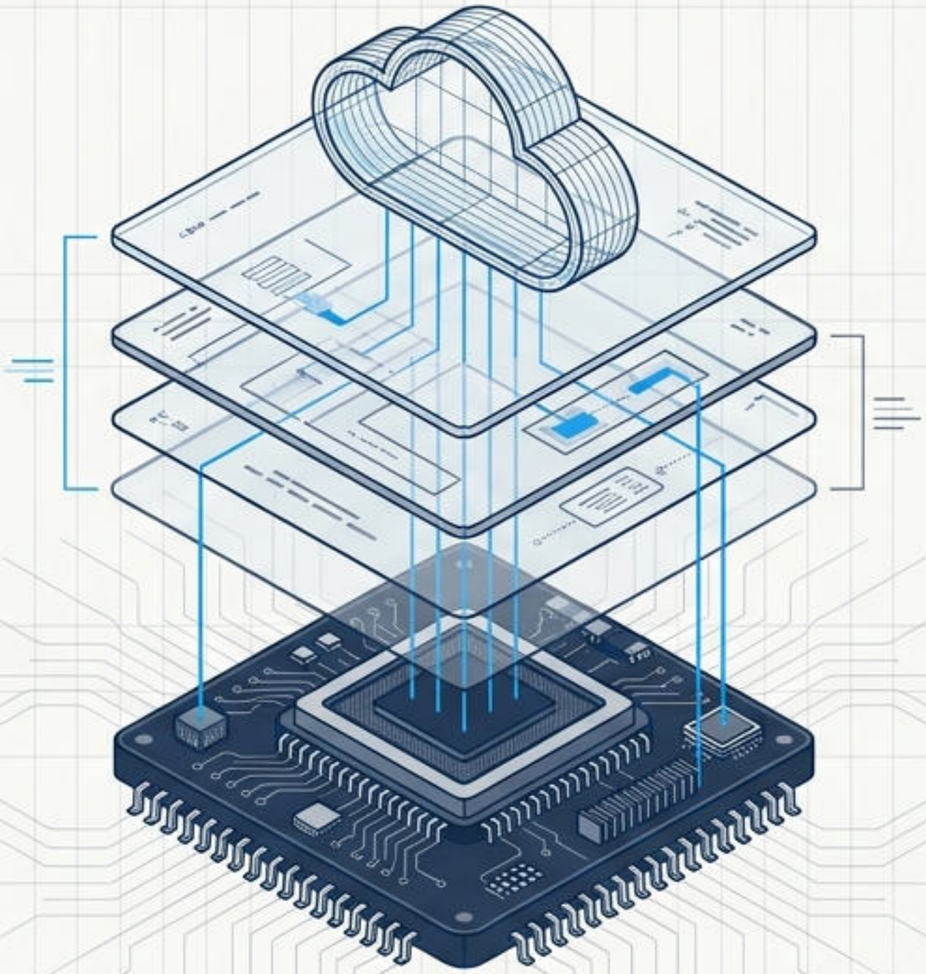
מערכות הפעלה: מהברזל ועד לענן

מדריך הסטודנט השלם: לינוקס, xv6, קונטיינרים וסביבות ענן (פרק 10)



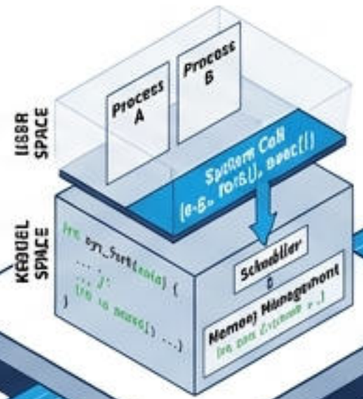
https://operating-system-5l3.pages.dev/

מפת דרכים לארכיטקטורת מערכת

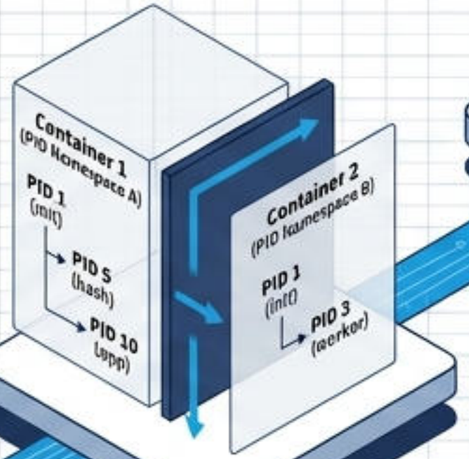
1. פני השטח:
לינוקס והטרמינל
(היכרות עם סביבת העבודה)



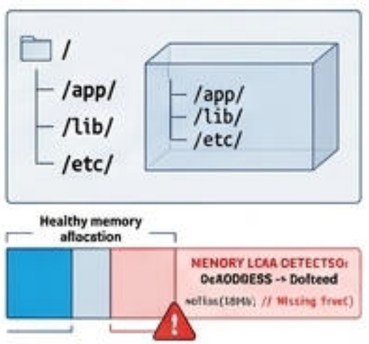
2. הליבה הפנימית:
מערכת xv6 וממ"ן 01
(הבנת הגרעין ו-
System Calls)



3. בידוד תהליכים:
קונטיינרים וממ"ן 02
(פתרון בעיית ה-PID
(PID Namespace)

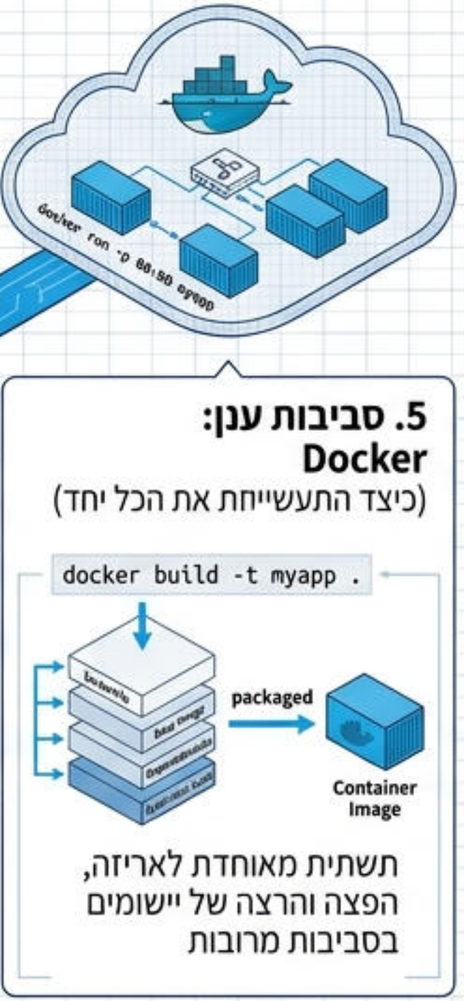


4. בידוד משאבים:
עגינה וממ"ן 03
(אשלית מערכת הקבצים וליפות
זיכרון)



מסהסות Memory
(אשליית מערכת הקבצים
ודליפות

5. סביבות ענן:
Docker
(כיצד התעשייחת את הכל יחד)

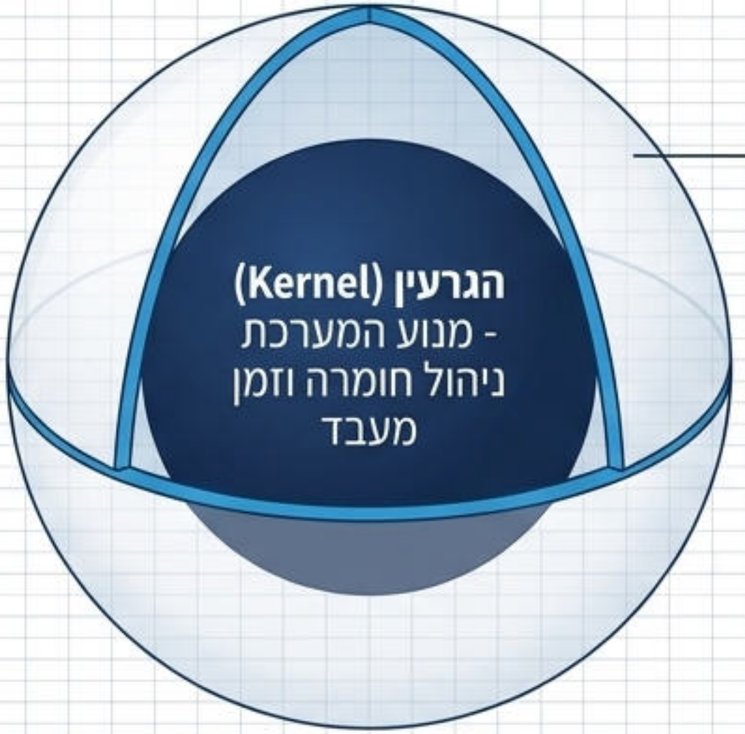


תשתית מאוחדת לאריזה,
הפצה והרצה של יישומים
בסביבות מרובות

2. הליבה הפנימית:
מערכת xv6 וממ"ן 01
(הבנת הגרעין ו-
System Calls)

מהי לינוקס?

מערכת הפעלה בקוד פתוח (Open Source), בעלת מבנה היררכי המתחיל משורש אחד (/) אחד (/), בניגוד לחלוקת הכוננים הכוננים ב-Windows.



משפחה	מאפיין מרכזי	קהל יעד
Debian & Ubuntu	יציבות וידידותיות (Ubuntu היא המערכת המארחת של הקורס)	למתחילים, מפתחים ושרתים
Red Hat & Fedora	הסטנדרט התאגידי וחדשנות מהירה (Fedora)	אנטרפרייז, בנקים ומפתחים
Arch Linux	שיטת הרכב-זאת-בעצמך ועדכונים רציפים (Rolling Release)	משתמשי לינוקס מתקדמים
Kali Linux	מאות כלי תקיפה מובנים מראש	אנשי אבטחת מידע והאקרים לבנים

הטרמינל: הלב הפועם של סביבת הפיתוח - שיחה ישירה עם מערכת ההפעלה.

ניהול קבצים

mkdir: יצירת תיקייה חדשה.
touch: יצירת קובץ טקסט ריק.
cp / mv: העתקה / הזזה (וגם שינוי שם).
rm: מחיקה לצמיתות (אין סל מיחזור).

ניווט (השילוש הקדוש)

pwd: הדפסת נתיב התיקייה הנוכחית המדויק.
ls: הצגת רשימת קבצים (**ls -l** להרשאות).
cd: מעבר תיקייה (**cd ..** חזרה רמה אחת אחורה).

ניהול תהליכים

ps: רשימת תהליכים נוכחיים וה-PID שלהם.
top: טבלת צריכת משאבים בזמן אמת (Task Manager).
kill <PID>: חיסול תוכנית תקועה לפי זהות.
man: ספר ההוראות המלא לכל פקודה.

הצצה לתוכן

cat: הדפסת כל תוכן הקובץ ברצף.
less: פתיחת קובץ ארוך לגלילה (יציאה עם q).
head / tail: הצגת 10 שורות ראשונות/אחרונות.

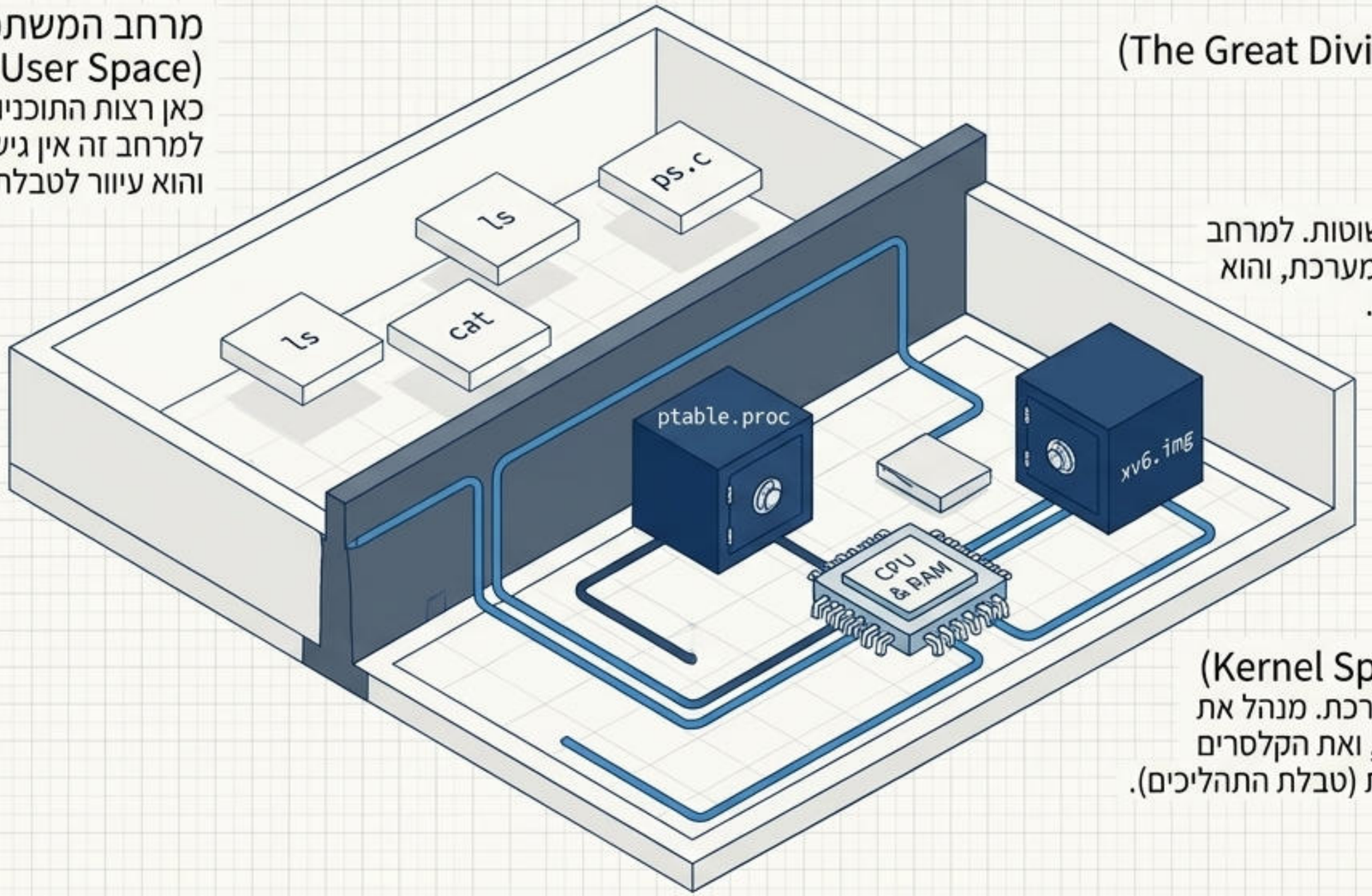
מערכת xv6: חזרה למקורות

לינוקס המודרנית מכילה 30 מיליון שורות קוד. חוקרי MIT פיתחו את xv6 כגרסה פשוטה וחינוכית (מבוססת ‹(Unix V6), הכוללת חוסרים ובאגים מכוונים המאלצים סטודנטים לפתח בעצמם.

הפרדת הרשויות (The Great Divide)

מרחב המשתמש (User Space)

כאן רצות התוכניות הפשוטות. למרחב זה אין גישה לזיכרון המערכת, והוא עיוור לטבלת התהליכים.



מרחב הגרעין (Kernel Space)

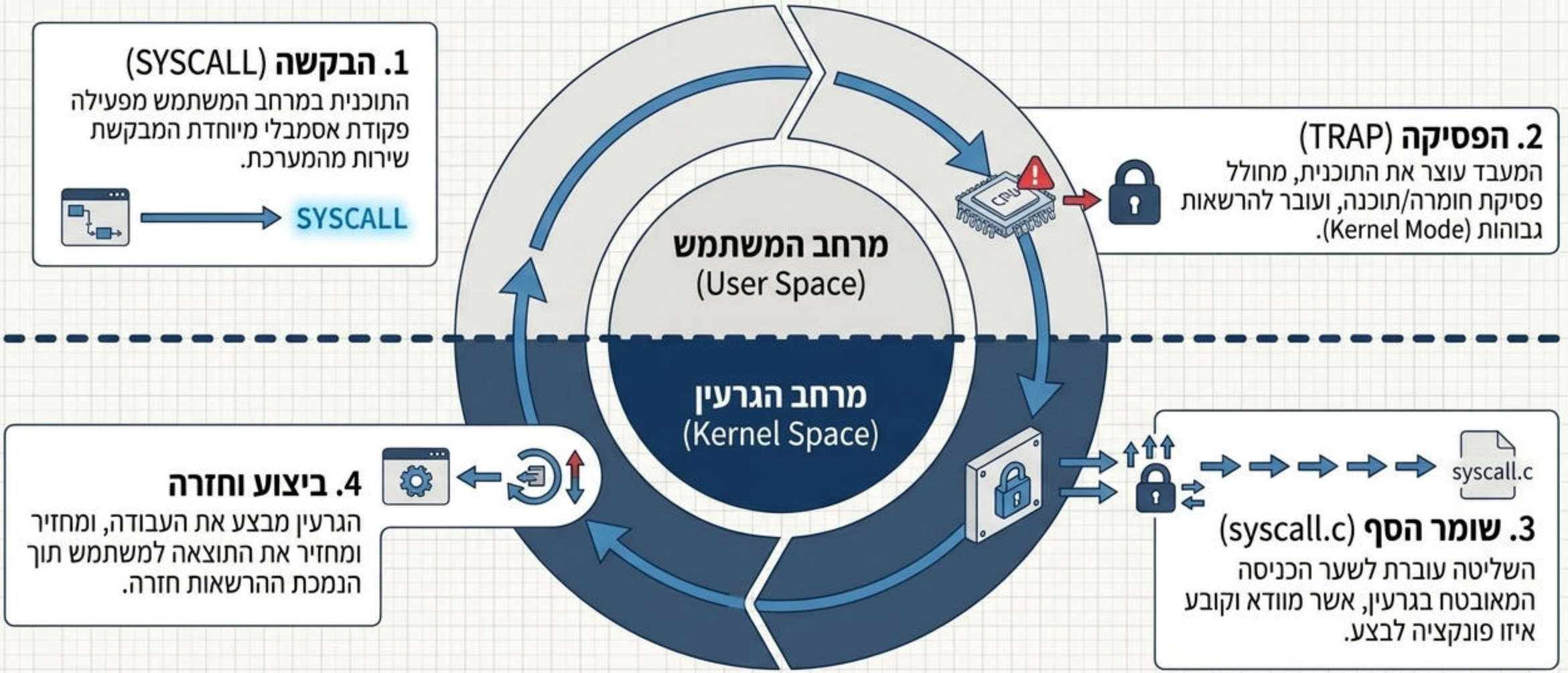
הלב המאובטח של המערכת. מנהל את הזיכרון, הגישה לחומרה, ואת הקלסרים הסודיים של כל המערכת (טבלת התהליכים).

מרחב המשתמש (User Space)

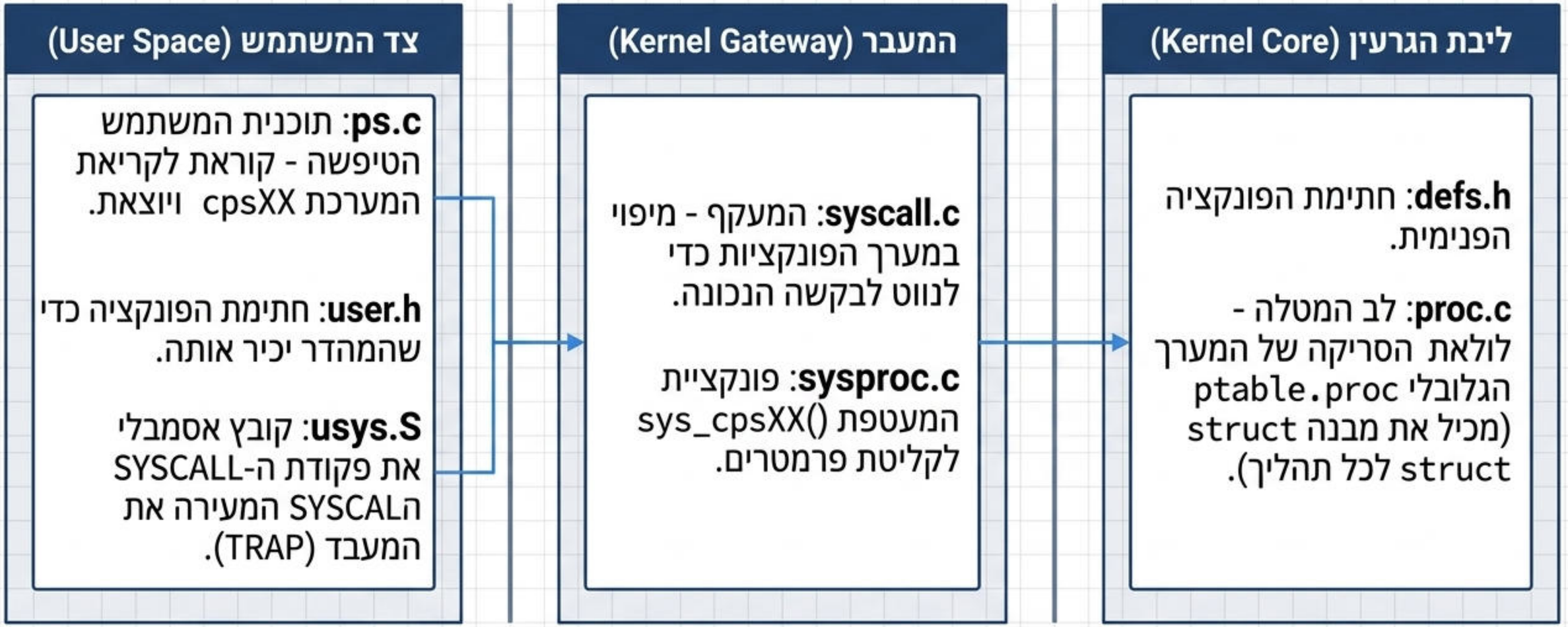
כאן רצות התוכניות הפשוטות. למרחב זה אין גישה לזיכרון המערכת, והוא עיוור לטבלת התהליכים.

חציית הגבול: קריאת המערכת (System Call) ומנגנון ה-TRAP

תוכנית אינה יכולה לקרוא לפונקציית גרעין ישירות. היא חייבת לבקש יפה במסלול מאובטח:



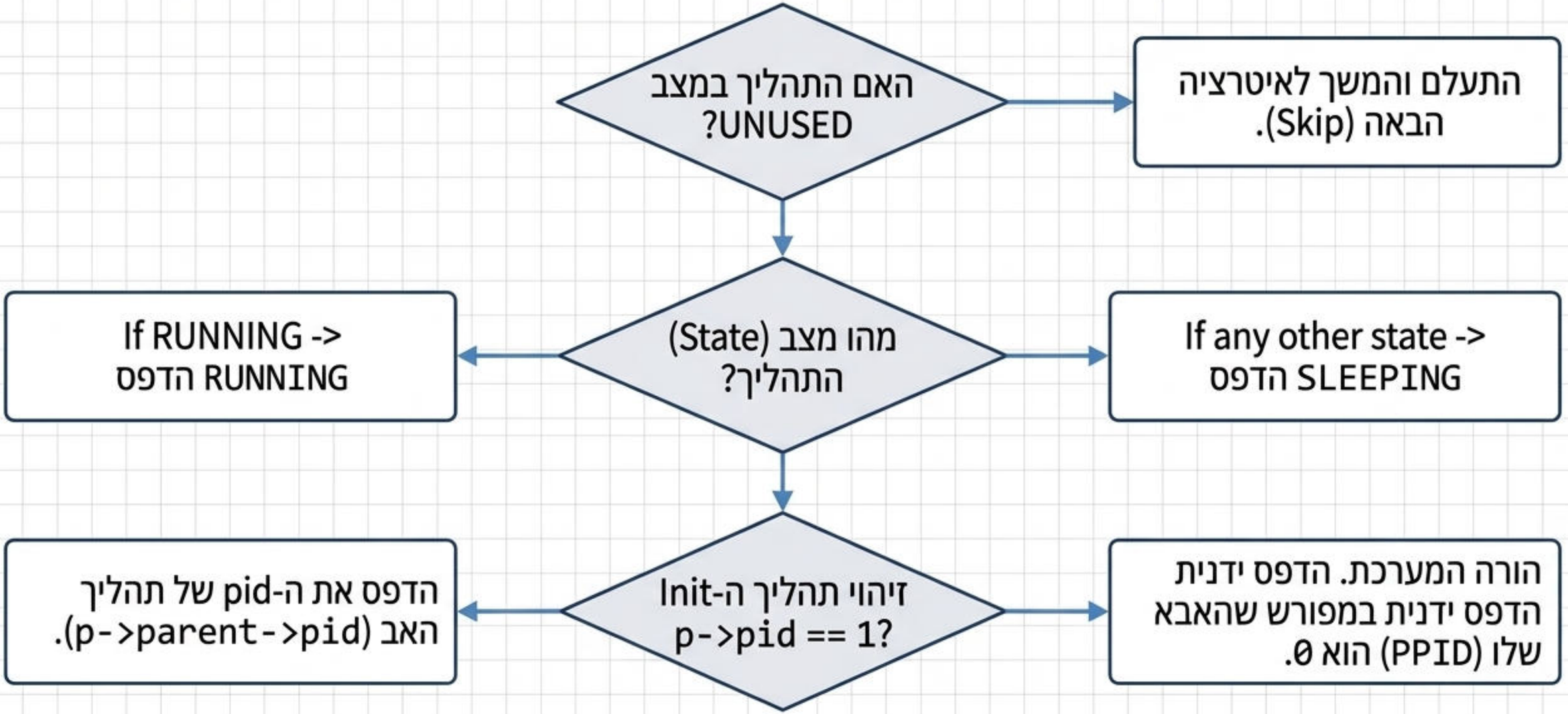
ממ"ן 01: ניתוח לב פתוח ליישום הפקודה ps. כדי לחלץ את טבלת התהליכים, יש לעדכן 8 קבצים לאורך כל המסלול:



קימפול: הוספת _ps_ לבלוק ה-UPROGS בקובץ ה-Makefile כדי שמערכת הבנייה תכניס אותה ל-fs.img.

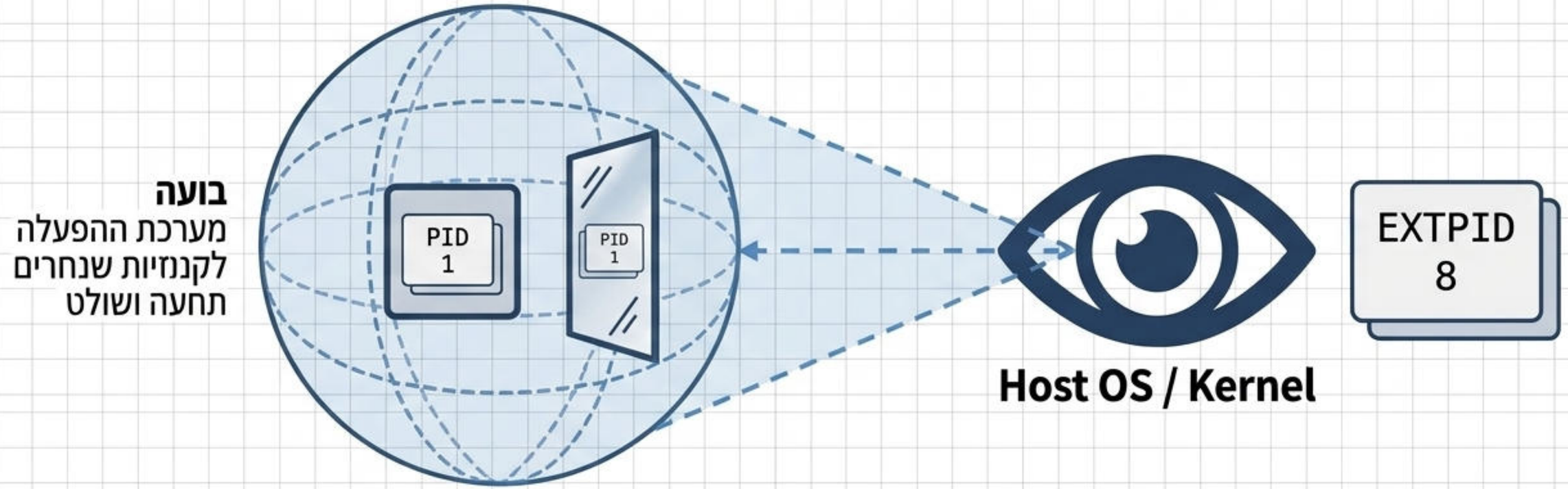
החוקים העסקיים של proc.c (ליבת המטלה)

בתוך לולאת הסריקה, המערכת נדרשת להדפיס נתונים (pid, name, sz, parent) תוך אכיפת חוקים קשיחים:



ממ"ן 02: קונטיינרים ובעיית הזהות הכפולה (PID Namespace)

מעבר ממערכת שטוחה בה כולם רואים את כולם, למערכת של קונטיינרים.
קונטיינר אינו חפץ ממשי, אלא בועה של בידוד שהגרעין מייצר סביב תהליך כדי לגרום לו לחשוב שהוא שולט במחשב לבדו.



אשליית תעודות הזהות

מערכת ההפעלה נאלצת מערכת ההפעלה נאלצת לשקר לתהליך ולנהל עבורו זהות כפולה:

PID (זהות פנימית): התהליך הראשון שרץ בתוך הבועה (למשל sh) בטוח לחלוטין שהמזהה שלו הוא 1.

EXTPID (זהות חיצונית): המזהה האמיתי כפי שמערכת ההפעלה המארחת וזן רואים אותו מבחוץ - מזהה גלובלי וייחודי (למשל, 8).

הבאג הלוגי: מדוע הקוד מממ"ן 01 נכשל?

נניח שתהליך 7 יצר קונטיינר והריץ בתוכו את sh.
ל-sh הזה יש EXTPID=8 וזהות פנימית PID=1.

✗ הבאג מממ"ן 01

```
if (p->pid == 1)
    ppid = 0;
```

הטעות: בעולם של קונטיינרים, לכל קונטיינר יש תהליך שחושב שהוא PID ! הקוד הקודם יזהה בטעות את ה-sh של הקונטינר כ-init הראשי, וידפיס שהאבא שלו הוא 0 במקום 7 (המזהה של תהליך 7 שיצר אותו).

✓ הדרך הכשרה (MMN 02)

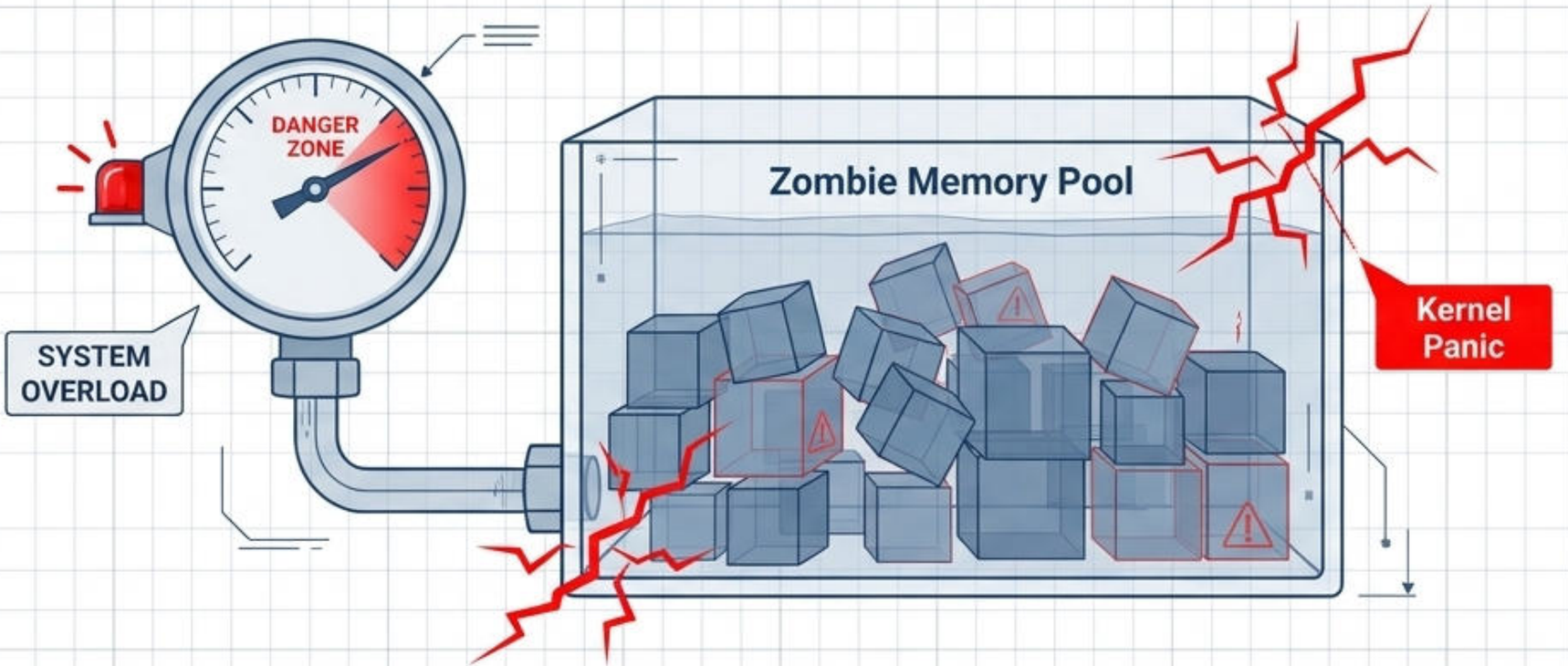
```
if (p->extpid == 1)
    ppid = 0;
```

התיקון: כמפתחי Kernel, עלינו להסתמך על האמת האובייקטיבית בלבד. רק התהליך שהזהות וחיצונית (EXTPID) שלו היא 1 הוא ה-init המקורי של המחשב.

(הערה: בממ"ן 02 מעדכנים את syscall.h רשמית, ומוסיפים עמודת זמן מעבד cputime).

ממ"ן 03: מרחבי שמות לעגינה (Mount Namespaces) ודליפת המשאבים

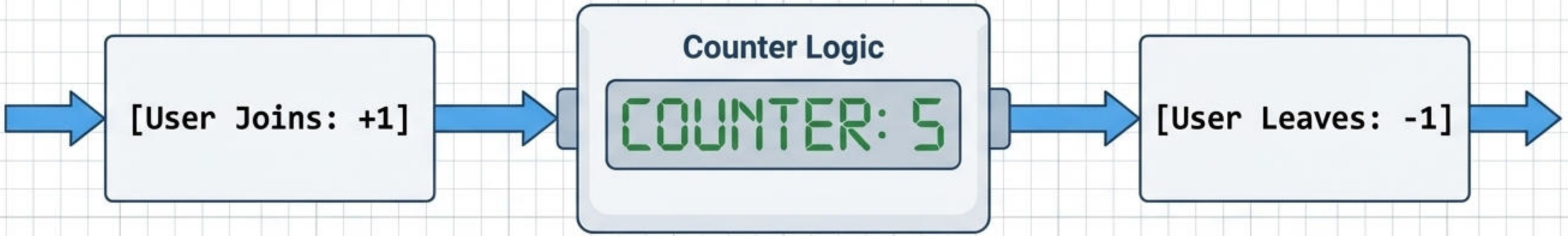
בממ"ן 03 מרחיבים את הבידוד: הקונטיינר מקבל אשליה של מערכת קבצים וירטואלית פרטית משלו.
האובייקטים המנהלים עגינה זו נלקחים ממאגר מוגבל בזיכרון הליבה.



<u>הבאג המכוון (Resource Leak):</u>
במערכת ה-xv6 של המטלה נשתל באג - כאשר קונטיינר מסיים את ריצתו, המערכת שוכחת לנקות אחריה.
אובייקטי העגינה לא מוחזרים למאגר אלא נשארים תפוסים בזיכרון כזומבים.
התוצאה? בהרצה השנייה של תוכנית הבדיקה mounttest, המאגר מתמלא לחלוטין, והמערכת קורסת לחלוטין.

הפתרון הרעיוני: ספירת מלאי (Reference Counting)

מכיוון שמספר תהליכים עשויים לחלוק קונטיינר, מנהלים מונה משתמשים.
יש לנקות משאבים רק כשהמונה מגיע ל-0.



יישום בפונקציה mount_nsleave (קובץ mnt_ns.c):

[Step 1] הפחתה: ניגשים לאובייקט ה-mount namespace של התהליך ומורידים את המונה שלו ב-1.

```
mnt_ns->ref_count--;
```

[Step 2] תנאי: משתמשים במשפט if כדי לבדוק האם המונה שווה כעת ל-0 (כלומר, האם אני התהליך האחרון האחרון שעוזב?).

```
if (mnt_ns->ref_count == 0) {
  ...
}
```

[Step 3] ניקוי: רק בתוך בלוק ה-if, קוראים לפונקציית העזר המרוקנת ומשחררת את המשאבים בחזרה למאגר.

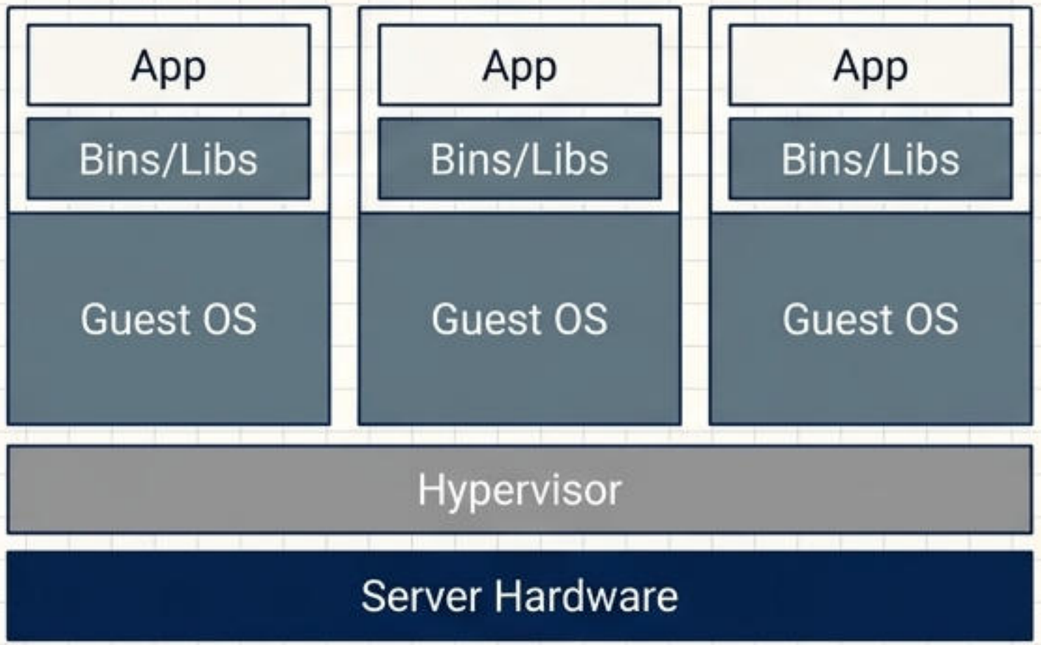
```
free_mnt_ns(mnt_ns); // Resources Released
```

(הערת הגשה: בממ"ן זה מגישים רק את קובץ ה-.c וקובץ ה-Dockerfile העיוני).

המהפכה של דוקר (Docker vs. VMs)

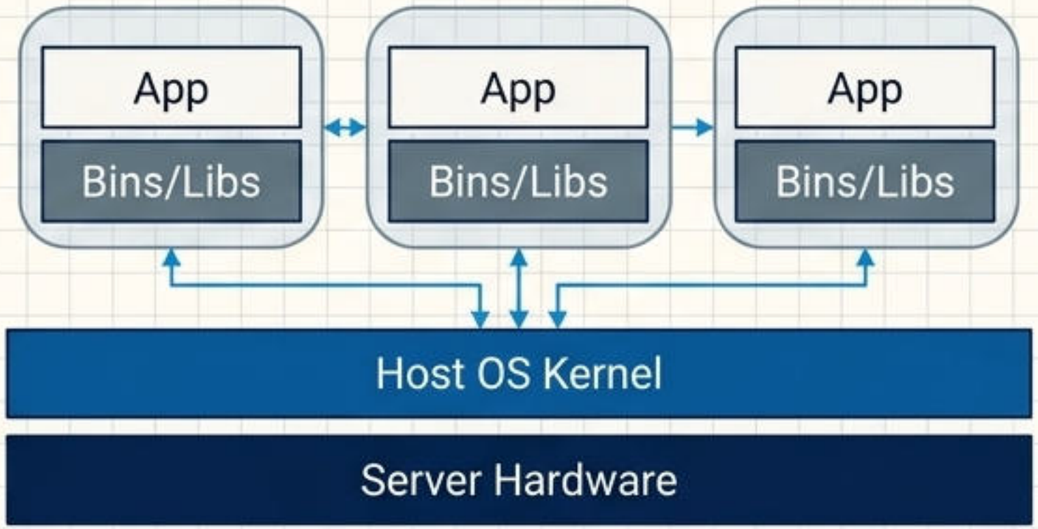
מדוע התעשייה נטשה את המכונות הוירטואליות לטובת **וירטואליזציה** ברמת מערכת ההפעלה?

מכונות וירטואליות (VMs) - השיטה הישנה



מריצות עותק מלא, כבד וזללן-משאבים של מערכת הפעלה שלמה (Guest OS) עבור כל אפליקציה.

המהפכה - Docker



קונטיינרים קלים ומהירים העולים בשבריר שנייה. הם חולקים גרעין (Kernel) יחיד תוך שמירה על בידוד מוחלט באמצעות Namespaces (מחיצות לוגיות) ו- Cgroups (הגבלת צריכת CPU וזיכרון).



[build]



[run]

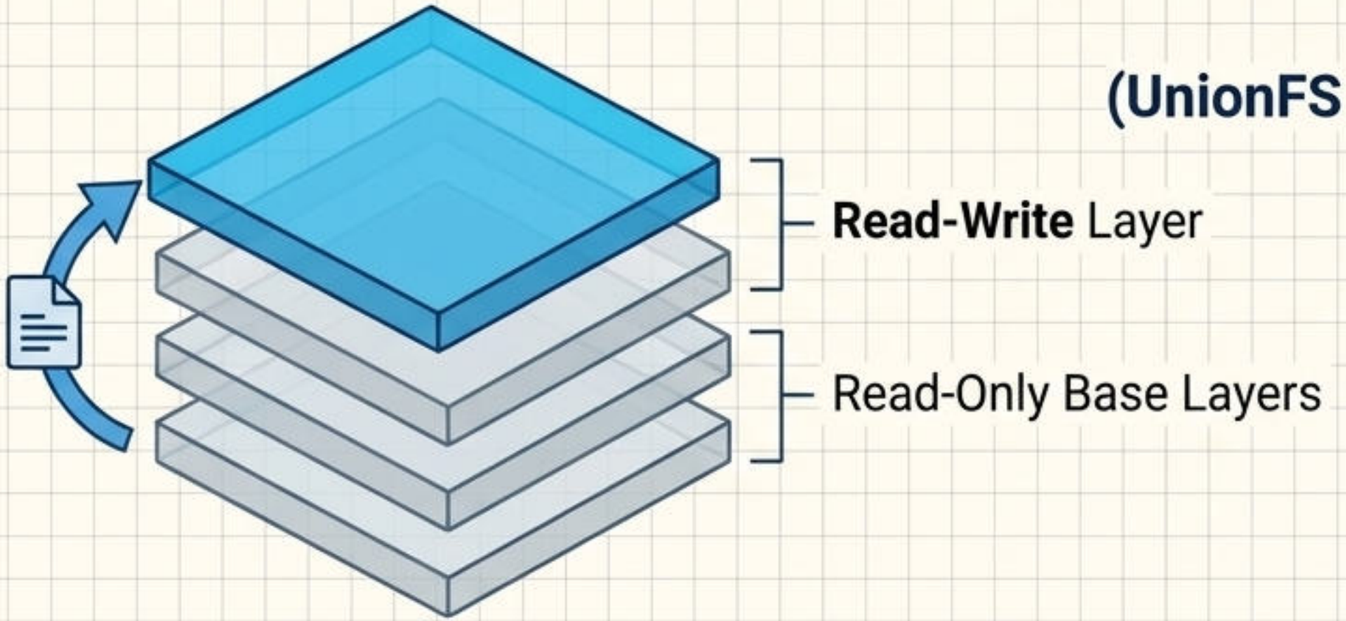


Dockerfile (המתכון)
קובץ טקסט עם הוראות בנייה.
תחביר: FROM, RUN, COPY, CMD

Image (התבנית)
תוצר לקריאה בלבד
(Read-Only)

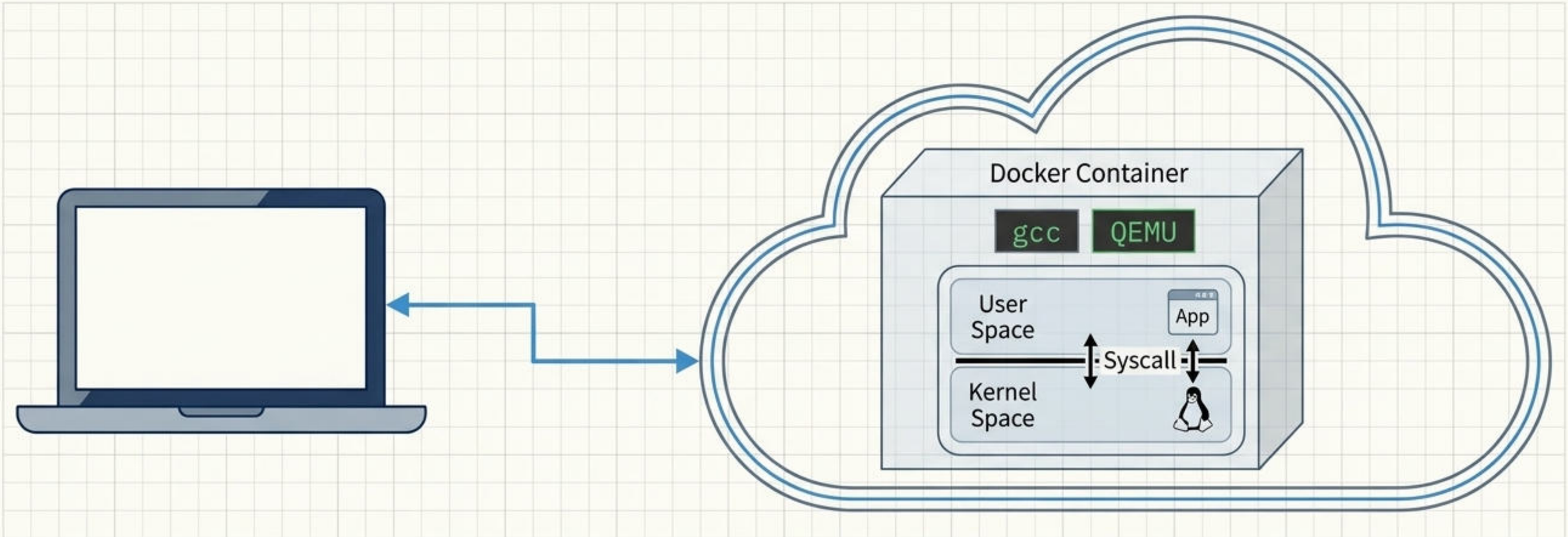
Container (המכונה החיה)
המופע החי. דוקר מניח שכבה
דקיקה לכתיבה (Read-Write)

מעגל החיים ושכבות (UnionFS & Copy-On-Write)



מנגנון CoW: רק כאשר קונטיינר מנסה לשנות קובץ **קיים**, דוקר **מעתיק אותו בסתר** מהשכבה התחתונה לעליונה הייחודית לו, ורק שם מבצע את השינוי. יעוכותיה **יעילות מקסימלית**.

תמונת הסיכום: למה כולם עוברים לענן? (GitHub Codespaces)



הכל מתחבר לדרך שבה אתם לומדים בקורס הזה ממש. מדוע לא ביקשו מכם להתקין תוכנות כבדות על המחשב בבית?
כי התעשייה עברה לענן.
GitHub Codespaces משתמש מאחורי הקלעים ב-Docker Image כדי לשלוף ולהריץ באופן מיידי Container מבודד המכיל Linux **ואת כל הכלים הדרושים לקורס** (gcc , QEMU). זה מאפשר סביבת פיתוח מהירה, נטולת זבל, עם אפס התקנות מקדימות. כל **מה שלמדתם** בפרקים הקודמים – הוא מה שמאפשר לסביבת הלמידה הנוכחית שלכם להתקיים.